Smart Traffic Management System

Final Project Report (IA-2): Route Navigation using A* extended with CSP, Rule-Based Reasoning and Responsible AI

Dhilfa A | Register No. VH15152 | CSE (AIML), II Year / III Sem / Sec A | Batch 7
Course: 25ML35T Foundations of Artificial Intelligence | Faculty: Dr. Manoj Kumar D S
Repository: github.com/dhilfa082-tech/Route-Navigation-A- | Final tag: v2.0

1. Problem Formulation

Phase 1 (IA-1) found the lowest-cost route between two places on a weighted road graph using A* with a straight-line-distance heuristic, and compared it with BFS. Phase 2 extends the same project into a small smart-traffic system. Real traffic is dynamic: roads get blocked, weather slows vehicles, congestion is uncertain and signals must share limited green time. The system therefore needs to (a) plan signal timings, (b) react to incidents by changing edge costs and re-planning, and (c) choose routes by expected travel time while staying fair and explainable.

Element	Definition
States	The junction (node) the vehicle is at, on the current road map.
Actions	Drive along an open road (edge) from the current node to a neighbouring node.
Goal test	Current node equals the destination node.
Path cost	Sum over edges of (travel time x congestion factor) + signal delay at the junction entered (+ a fairness penalty on school and residential roads when routing).

2. Approach

Phase 1 code is kept unchanged. Three layers are built on top and joined by *src/main.py*:

- **CSP layer** chooses green times at 3 junctions (backtracking, then backtracking + forward checking).
- **Knowledge and reasoning layer** is a forward-chaining rule base. Derived facts change edge costs, then A* re-plans on the new graph.
- **Applied and responsible AI layer** uses hour-of-day congestion probabilities, routes by minimum *expected* time, a fairness penalty for sensitive zones, and plain-English explanations.

A* is used with f = g + h. The heuristic h is the straight-line distance. It stays admissible on the dynamic graph because every change makes an edge cost larger or removes the edge (blocked road, slower speed, expected congestion at least 1 times free flow, signal delay at least 0). A green corridor only removes signal delay, never travel time. So h never overestimates and A* remains optimal. This is also checked experimentally against Dijkstra (Section 7).

3. CSP Formulation: Signal Timing

Part	Definition
Variables	Green time of each phase at each junction: J1_NS, J1_EW, J1_TURN, ... (3 junctions = 9 variables).
Domains	{15, 20, 25, ..., 60} seconds (10 values). The lower bound 15 s is the pedestrian minimum.
Constraints	(1) Fixed cycle: NS + EW + TURN + 5 s lost time = 90 s. (2) Pedestrian minimum: every green >= 15 s. (3) Priority: a phase with higher demand gets green >= a phase with lower demand. (4) No conflicting greens: phases run one after another, never together.

Solvers. Plain backtracking checks constraints only after an assignment. Backtracking with forward checking removes values from the other variables of the same junction that can no longer satisfy the constraints, and backtracks early when a domain becomes empty. Demand values are random but seeded with the register number 15152.

Junctions	Plain: assignments tried	Forward checking: assignments tried	Plain time (s)	FC time (s)
2	222	26	0.00018	0.00019
3	313	37	0.00026	0.00032
4	404	48	0.00034	0.00041

Table 1: Plain backtracking vs forward checking (one run; times vary slightly between runs).

Forward checking tried about 8 times fewer assignments (for example 37 vs 313 for 3 junctions). The runtime difference is tiny because the problem is very small, so the number of assignments is the fair measure.

Resulting plan for Junction 1: NS = 30 s (demand 54), EW = 30 s (demand 30), TURN = 25 s (demand 10); 30 + 30 + 25 + 5 = 90 s.

4. Knowledge Base and Inference Trace

Facts are stored as tuples, for example *("Accident", ("B","D"))*, *("Rain",)*, *("Night",)*. The rule base is:

Rule	Meaning
R1: Accident(e) -> Blocked(e)	A road with an accident is removed from the graph.
R2: Rain AND Night -> SpeedFactor(0.7)	Travel times are divided by 0.7 (vehicles move slower).
R3: EmergencyVehicle(v) AND RouteOf(v, edges) -> GreenCorridor(e)	Signal delay on every road of the vehicle's route is set to 0.

Forward chaining repeatedly fires every rule whose conditions hold until no new fact appears. **Inference trace** for the facts Accident(B,D), Rain, Night:

```
R1 Accident(e) -> Blocked(e)  =>  derived ('Blocked', ('B', 'D'))
R2 Rain ^ Night -> SpeedFactor(0.7)  =>  derived ('SpeedFactor', 0.7)
```

The derived facts update the edge costs and A* re-plans. Normal route A-B-D-F (cost 18.00) becomes A-C-E-F (cost 29.25) after the accident and rain at night. With an ambulance on that route, rule R3 derives GreenCorridor for A-C, C-E and E-F and the cost drops to 25.25.

5. Uncertainty Model

Each road is in one of three congestion levels with a travel-time multiplier: LOW (1.0), MEDIUM (1.5), HIGH (2.2). The probability of each level depends on the hour of the day, and each road has a sensitivity s (0.5 calm, 1.0 normal, 1.5 jam-prone) so that multiplier = 1 + s x (level multiplier - 1). Expected travel time of a road = free-flow time x sum over levels of P(level) x multiplier. A* then runs on these expected times and returns the route with the minimum expected travel time.

Period	P(LOW)	P(MEDIUM)	P(HIGH)
Peak (08-10, 17-19)	0.10	0.30	0.60
Normal daytime	0.50	0.35	0.15
Night (22-05)	0.80	0.15	0.05

6. Complexity and Practical Limits

Component	Time	Space	Practical limit
A* (binary heap)	O(b^d) worst case; O(E log V) on this graph	O(V)	Fine for the 6-node demo map; a real city needs a much better heuristic.
CSP backtracking	O(d^n) per junction (d = 10, n = 3: at most 1000 combinations)	O(n)	Junctions are independent, so cost grows linearly with the number of junctions.
Forward checking	Same worst case, far fewer nodes in practice	O(n x d)	Extra work per assignment, repaid by earlier pruning.
Forward chaining	O(R x F) per pass (R rules, F facts)	O(F)	Tiny rule base here.

Component	Time	Space	Practical limit
Expected-time graph	O(E) per query hour	O(E)	Recomputed when the hour changes.

7. Results

Simulation setup. 100 random trips (random start, goal and hour, seed 15152). On about 20% of the trips an accident blocks a random road, and the rule engine updates the graph before A* runs. **Before**: every junction uses fixed timing 30 s / 30 s / 25 s. **After**: each junction uses the feasible CSP plan with the least average delay. Signal delay uses the uniform-delay formula d = 0.5 C (1 - g/C)^2 / (1 - (g/C) X).

Measure	Fixed timing	CSP timing
Average signal delay per junction	29.4 s	26.2 s
Average trip time (100 trips)	11.38 min	11.34 min
A* cost differs from Dijkstra	0 of 100	0 of 100

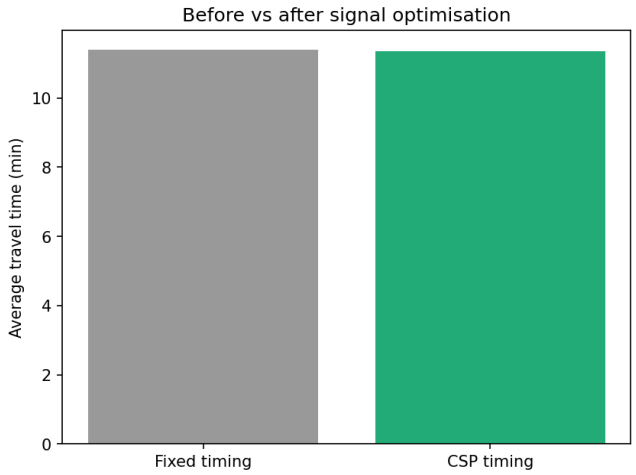

Figure 1: Average travel time before vs after signal optimisation.

Interpretation. The CSP plans cut average signal delay by about 11%, but total trip time improves only 0.4% because junction delay is a small share of an 11-minute trip. A* matched Dijkstra on every trip, including trips with blocked roads, so A* stayed optimal on the dynamic graph.

Effect of the fairness penalty (route from A to F):

Hour	Fairness OFF	Fairness ON
03:00	A-B-D-F, 21.04 min	A-B-D-F, 21.04 min
14:00	A-C-E-F, 24.82 min (crosses school and residential zone)	A-B-D-F, 25.99 min
08:00	A-C-E-F, 29.37 min (crosses school and residential zone)	A-B-D-F, 37.58 min
18:00	A-C-E-F, 29.37 min (crosses school and residential zone)	A-B-D-F, 37.58 min

Table 2: Route choice with and without protection of sensitive zones.

At peak hours protecting the zones makes the trip longer by about 8 minutes (37.58 vs 29.37). This is the cost of fairness and is a policy decision for a human operator. The 16 unit tests in the *tests* folder all pass.

8. Responsible AI

Explanation examples

```
Junction 1: cycle 90s, 5s lost to yellow/all-red, so 85s of green are shared.
  - NS: 30s green for demand 54 veh/min (never below the 15s pedestrian minimum).
  - EW: 30s green for demand 30 veh/min ...
  The busiest approach (NS) gets at least as much green as any less busy approach.

Route A -> B -> D -> F at 08:00, expected travel time 37.6 min.
It avoids school and residential zones.
A* with f = g + h found the minimum-cost path on the updated map.
```

Risks and bias checks

- **Fairness to residents.** Without protection, A* sends traffic through the school and residential roads (Table 2). The penalty removes this at the cost of longer trips. Ordinary roads carry that extra load, so their congestion should be monitored.
- **Fairness between approaches.** The pedestrian minimum of 15 s guarantees that no phase is starved.
- **Data bias.** Congestion probabilities and demand here are synthetic. Real data may under-represent some areas, which would give those areas worse signal plans and slower emergency access. Results should be checked per area before deployment.
- **Model risk.** The signal delay formula is simplified and the map is tiny, so results must not be used for real traffic control without validation.

Privacy, accountability and governance

- **Privacy.** Real systems use GPS traces. These should be anonymised, kept only as long as needed and used only for traffic purposes. This project uses only synthetic data.
- **Accountability.** Every route and signal plan can be explained by the rules that fired and the constraints used. A human operator must be able to override any rule or plan, and decisions should be logged for audit.
- **Governance and regulation.** Deployment would need approval from the traffic authority, compliance with data-protection law, safety review of signal plans (no conflicting greens, pedestrian time) and regular re-testing as traffic patterns change. Emergency-vehicle priority needs a strict, auditable trigger so it cannot be misused.

9. Reflection

- **Hard parts:** keeping A* optimal when rules change the graph (the admissible heuristic had to be protected), and making the CSP produce useful plans instead of just any feasible plan.
- **Honest result:** the improvement in total travel time is small (0.4%). The signal-delay reduction (29.4 s to 26.2 s) is the clearer benefit.
- **Improvements:** a larger real map, time-varying demand in the CSP, AC-3 propagation across junctions (coordinated green waves), learning congestion probabilities from data, and testing against real traces.

Honesty note: An AI assistant (Claude) helped me write and test the code and draft this report. I have read the code, can explain every part, and checked the results by running the program and the tests.

References: Russell and Norvig, Artificial Intelligence: A Modern Approach (A*, CSP, forward chaining); Webster's uniform-delay formula (simplified).